

R Functions You Probably Never Knew

Roy Mwavita

2026-06-30



Introduction

As data analysts and researchers, we often default to the most familiar functions, unaware that R's core library and specialized packages contain powerful, streamlined tools designed to eliminate repetitive coding patterns. This presentation highlights highly effective functions that turn multi-step tasks into double/single-line operations.

1. readxl::excel_sheets()

Case: You have a workbook with many tabs and must manually open Excel just to check the tab names before writing your import script.

Normally, you'd do this:

```
# Manually checking Excel, then hardcoding
data <- read_excel("data.xlsx", sheet = "Q1_2026_Final_Revised")
```

Here's where `readxl::excel_sheets()` comes in.

```
# Checking the sheets in the dataset
sheets <- excel_sheets("data.xlsx")
```

Let's take for instance this dataset:

```
# Loading the package
library(readxl)

# Fetching the url of the data
url <- "https://raw.githubusercontent.com/tidyverse/readxl/main/inst/extdata/clippy.xlsx"

# Downloading it working director
download.file(url, "clippy.xlsx", mode = "wb")

# Checking the sheets present
excel_sheets("clippy.xlsx")
```

```
## [1] "list-column"    "two-row-header"
```

Why it's better:

- Eliminates the need to manually open Excel
- Automates sheet name discovery for dynamic imports
- Prevents errors from hardcoded sheet names that may change
- Enables looping through multiple sheets programmatically

2. `base::all.equal()`

Case: You cleaned a dataset and want to verify it matches the source, but the standard `==` operator returns a massive, unreadable matrix of TRUE/FALSE values.

Normally you'd do this:

```
# Returns a confusing list of logicals for every single cell  
comparison <- df_cleaned == df_raw
```

Here's where `base::all.equal()` comes in:

```
# Returns a clear summary of differences or TRUE if identical  
diffs <- all.equal(df_cleaned, df_raw)  
print(diffs)
```

Why it's better:

- Returns a human-readable summary instead of a massive matrix
- Handles numeric tolerance for floating-point comparisons
- Provides detailed messages about what differs, not just that they differ
- Essential for regression testing and data validation pipelines

3. `base::findInterval()` & `cut()` – Smart Binning

Case: You need to categorize continuous data (e.g., age groups, program eligibility scores, vulnerability indices) into buckets. You write multiple `ifelse()` or `case_when()` statements that become unreadable.

Normally you'd do this:

```
df$age_group <- ifelse(df$age < 5, "Under 5",  
                      ifelse(df$age < 12, "Child",  
                              ifelse(df$age < 18, "Adolescent",  
                                      ifelse(df$age < 60, "Adult", "Senior"))))
```

Here's where `base::cut()` & `findInterval()` comes in:

```
# Option 1: cut() with labels (more readable)  
df$age_group <- cut(df$age,  
                   breaks = c(0, 5, 12, 18, 60, 120),  
                   labels = c("Under 5", "Child", "Adolescent", "Adult", "Senior"))  
  
# Option 2: findInterval() for numeric categories  
breakpoints <- c(0, 5, 12, 18, 60)  
df$age_category <- findInterval(df$age, breakpoints)
```

Why it's better:

- One line replaces multiple nested conditions
- Much easier to modify breakpoints later
- `findInterval()` is lightning fast for many breakpoints
- `cut()` automatically creates labeled factors
- Reduces errors from complex nested logic

4. `data.table::fcase()` – Fast Vectorized Multiple Conditions

Case: You have multiple conditions and want a fast, readable alternative to nested `ifelse()` or `case_when()`. If you're working with large datasets, this is a game-changer.

Normally you'd do this:

```
library(dplyr)
df$risk_category <- case_when(
  df$score < 30 ~ "High Risk",
  df$score < 60 ~ "Medium Risk",
  df$score < 80 ~ "Low Risk",
  TRUE ~ "No Risk"
)
```

Here's where `fcase()` from `data.table` comes in:

```
library(data.table)

# Convert to data.table for speed
setDT(df)

df$risk_category <- fcase(
  df$score < 30, "High Risk",
  df$score < 60, "Medium Risk",
  df$score < 80, "Low Risk",
  default = "No Risk"
)
```

Why it's better:

- Significantly faster than `case_when()` on large datasets
- Cleaner syntax with default argument
- Vectorized for maximum performance
- Seamlessly integrates with `data.table` workflows
- Perfect for production code with big data

5. janitor::get_dupes() – Find Duplicates Instantly

Case: You suspect duplicate records in your dataset but need a quick way to identify them without writing complex grouping code.

Normally you'd do this:

```
# Manual duplicate detection
duplicates <- children %>%
  group_by(child_id) %>%
  filter(n() > 1) %>%
  arrange(child_id)

# Or even more manual
duplicate_ids <- children$child_id[duplicated(children$child_id)]
duplicates <- children[children$child_id %in% duplicate_ids, ]
```

Here's where `get_dupes()` comes in:

```
library(janitor)

# Find duplicates by child_id
duplicates <- children %>%
  get_dupes(child_id)

# Find duplicates by combination of columns
duplicates <- children %>%
  get_dupes(child_id, household_id, dob)

# Get summary of duplicates
duplicates %>%
  select(child_id, household_id, dob, dupe_count) %>%
  arrange(desc(dupe_count))
```

Why it's better:

- One line replaces complex grouping code
- Automatically adds a `dupe_count` column
- Shows you exactly which rows are duplicated
- Much faster than manual grouping with `dplyr`
- Can check multiple columns at once

6. janitor::tabyl() – Better Frequency Tables

Case: You need to create frequency tables with percentages, but `table()` gives you raw counts and `prop.table()` requires multiple steps. You waste time calculating percentages and formatting them for reports.

Normally you'd do this:

```
# Base R approach - multiple steps
counts <- table(df$program_area)
percentages <- prop.table(counts) * 100
result <- cbind(counts, percentages)
print(result)

# Or with dplyr - still multiple steps
df %>%
  group_by(program_area) %>%
  summarise(
    n = n(),
    percent = n() / nrow(df) * 100
  )
```

Here's where `janitor::tabyl()` comes in:

```
library(janitor)

# One-line frequency table with percentages
tabyl(df, program_area)

# Two-way cross-tabulation
tabyl(df, program_area, gender)

# With totals and percentages
tabyl(df, program_area, status) %>%
  adorn_totals(c("row", "col")) %>%
  adorn_percentages("row") %>%
  adorn_pct_formatting(digits = 1) %>%
  adorn_ns()
```

Why it's better:

- One line replaces multiple steps
- Handles 1-way, 2-way, and 3-way tables
- Pipes cleanly with dplyr
- Built-in formatting options (adorn_* functions)
- Perfect for M&E reporting and presentations
- Output is already formatted for printing

7 cleaner::clean_numeric() – Smart Numeric Cleaning

Case: You have messy text fields with numbers embedded (e.g., “\$12,345.67”, “qwerty123456”, “Positive (0.143)”) and need to extract the numeric values.

Normally you’d do this:

```
# Manual cleanup with regex
clean_number <- function(x) {
  x <- gsub("[^0-9.,-]", "", x) # Keep only numbers, dots, commas, dashes
  x <- gsub(",", "", x)       # Remove commas
  as.numeric(x)
}

# Apply to each column
df$income <- clean_number(df$income)
df$score <- clean_number(df$score)
```

Here’s where clean_numeric() comes in:

```
library(cleaner)
```

```
## Warning: package 'cleaner' was built under R version 4.5.3
```

```
# Handles multiple formats automatically
clean_numeric(c("$ 12,345.67", "€ 12.345,67", "12,345.67", "12345,67"))
```

```
## [1] 12345.67 12345.67 12345.67 12345.67
```

```
clean_numeric("qwerty123456")
```

```
## [1] 123456
```

```
clean_numeric("Positive (0.143)")
```

```
## [1] 0.143
```

Why it's better:

- Handles multiple formats automatically
- Understands usage of dots and commas in different languages
- Much cleaner than writing your own regex
- Perfect for cleaning financial or survey data
- Handles text mixed with numbers intelligently

8 cleaner::clean_Date() – Any Date Format

Case: Dates come in countless formats (Excel serial numbers, “13jul18”, “12-06-2012”, “14 August 2010”) and you waste time parsing them.

Normally you’d do this:

```
# Try multiple formats with lubridate
library(lubridate)
parse_date <- function(x) {
  if (is.numeric(x)) return(as.Date(x, origin = "1899-12-30"))
  tryCatch(ymd(x),
            error = function(e) tryCatch(dmy(x),
            error = function(e) tryCatch(mdy(x),
            error = function(e) NA)))
}
```

Here’s where clean_Date() comes in:

```
library(cleaner)

# Clean dates from various formats
clean_Date("13jul18", "ddmmmyy")
```

```
## [1] "2018-07-13"
```

```
# (assuming format 'dd-mm-yyyy')
clean_Date("12-06-2012")
```

```
## (assuming format 'dd-mm-yyyy')
```

```
## [1] "2012-06-12"
```

```
# (assuming format 'dd mmmm yyyy')
clean_Date("14 August 2010")
```

```
## (assuming format 'dd mmmm yyyy')
```

```
## [1] "2010-08-14"
```

```
# (assuming Excel format)  
clean_Date(38071)
```

```
## (assuming Excel format)
```

```
## [1] "2004-03-25"
```

Why it's better:

- Handles Excel serial numbers automatically
- Supports multiple date formats in one call
- Even understands Excel date format language (like d-mmm-yyyy)
- Perfect for cleaning data from multiple sources
- Eliminates complex `tryCatch()` parsing logic

Quick Reference Summary

Function	Package	Use Case
<code>excel_sheets()</code>	<code>readxl</code>	List Excel sheet names
<code>all.equal()</code>	<code>base</code>	Compare objects
<code>cut()</code> / <code>findInterval()</code>	<code>base</code>	Bin continuous data
<code>fcase()</code>	<code>data.table</code>	Fast multiple conditions
<code>get_dupes()</code>	<code>janitor</code>	Find duplicates
<code>tabyl()</code>	<code>janitor</code>	Frequency tables with percentages
<code>clean_numeric()</code>	<code>cleaner</code>	Extract numbers from text
<code>clean_Date()</code>	<code>cleaner</code>	Parse any date format

References

- R Core Team (2024). R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/> (<https://www.R-project.org/>)
- Wickham H, Bryan J (2023). readxl: Read Excel Files. R package version 1.4.3. <https://CRAN.R-project.org/package=readxl> (<https://CRAN.R-project.org/package=readxl>)
- Firke S (2023). janitor: Simple Tools for Examining and Cleaning Dirty Data. R package version 2.2.0. <https://CRAN.R-project.org/package=janitor> (<https://CRAN.R-project.org/package=janitor>)
- Dowle M, Srinivasan A (2023). data.table: Extension of data.frame. R package version 1.14.10. <https://CRAN.R-project.org/package=data.table> (<https://CRAN.R-project.org/package=data.table>)
- Hahsler M, Wieden S (2023). cleaner: Fast and Easy Data Cleaning. R package version 1.3.2. <https://CRAN.R-project.org/package=cleaner> (<https://CRAN.R-project.org/package=cleaner>)